



Level 0x0d

Engineering Degrees

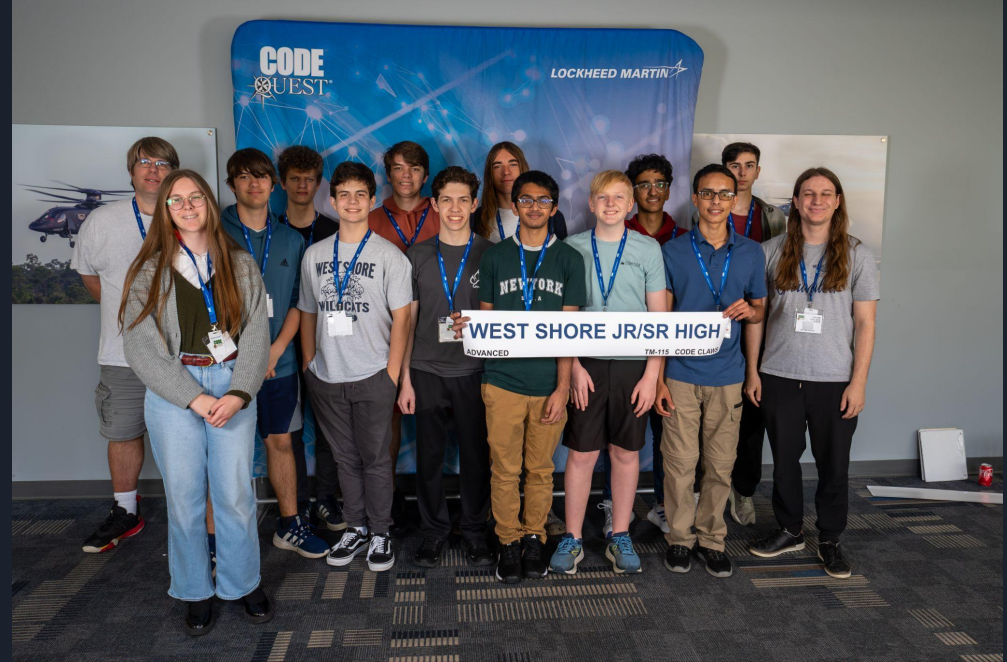


Topics

- Events
- Code Quest

Cyber Quest and Code Quest

- Code Quest
 - Saturday, Feb 28th
- Spring Break
 - March 23rd - 27th
- Cyber Quest
 - Saturday, March 28th
 - Registration Jan 5th -





Code Quest

Wildcat 1 (Advanced)

Status: Confirmed

Students: Samuel S - Eshan V - Jay J

Alternate Coach: Debra J

[Edit](#)[Assign Students](#)[Withdraw](#)

Wildcat 4 (Advanced)

Status: Confirmed

Students: Parker W - Zachary W - Charles L

Alternate Coach: Debra J

[Edit](#)[Assign Students](#)[Withdraw](#)

Wildcat 2 (Advanced)

Status: Confirmed

Students: Ian B - Nathan d - Jasper B

Alternate Coach: Debra J

[Edit](#)[Assign Students](#)[Withdraw](#)



Code Quest Schedule

7:00 am - 8:50 am	Registration, Breakfast, Setup
<i>7:30 am - 7:45 am</i>	<u>WESTSHORE ARRIVES!!!!</u>
8:50 am - 9:30 am	Welcome and Competition Rules
9:30 am - 12:00 pm	Competition (Wildcats DOMINATE!)
12:00 pm - 2:20 pm	Lunch and Activities
2:20 pm - 3:00 pm	Awards and Photos
3:00 pm	Swag Bags and Pick-up



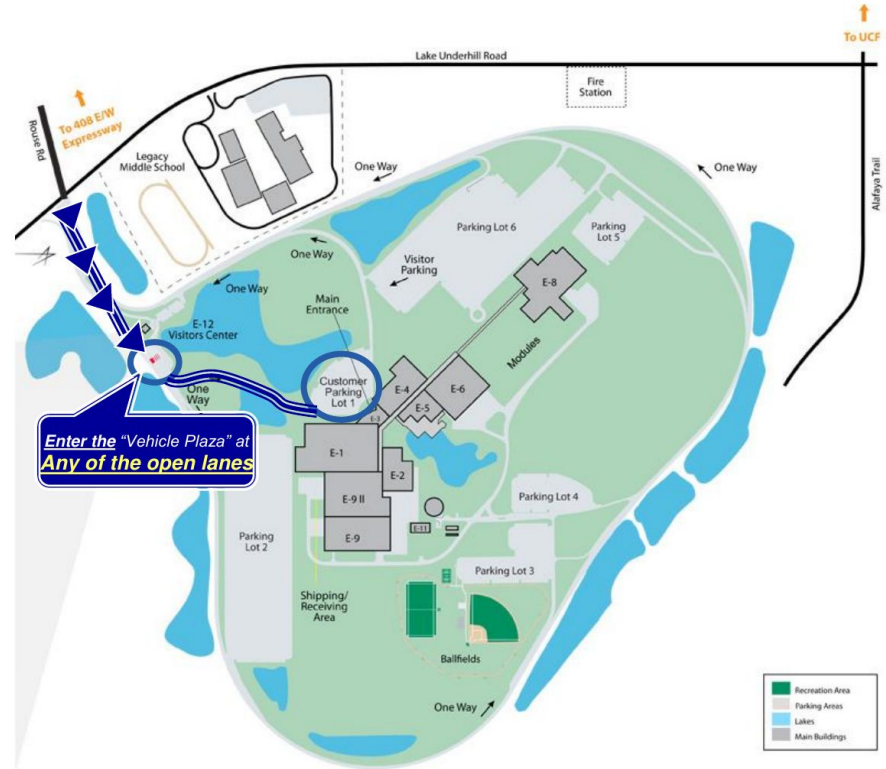
Security Notices

- At Security Gate, say code word “Code Quest.” (Please inform bus drivers, parent drop off/pick up, coaches/students driving to competition).
- Photo IDs are required for all coaches and students.
 - Coaches, non-U.S. students, and Permanent Residents are required to bring valid government-issued photo I.D. and original documentation (green card, passport, visa, resident card).
 - All students should bring a valid form of I.D. (school I.D. is acceptable).
- All phones must be left in your cars during the Code Quest event. Under no circumstances are any phones or recording devices allowed inside Lockheed Martin facilities. This applies to coaches and students.
- In case of emergency, family can call/text Michelle Begonia (321-230-5695) or Abbie Mitchell (407-446-9871) on the day of the competition.





- Drive onto Lockheed Martin Facility at Rouse Rd & Lake Underhill
- Enter the “Vehicle Plaza” at any of the open lanes.
- Stop & Check in with Security Guard who will grant you access to the facility.
- Parking - Follow road on the LEFT for Parking Lot #1.





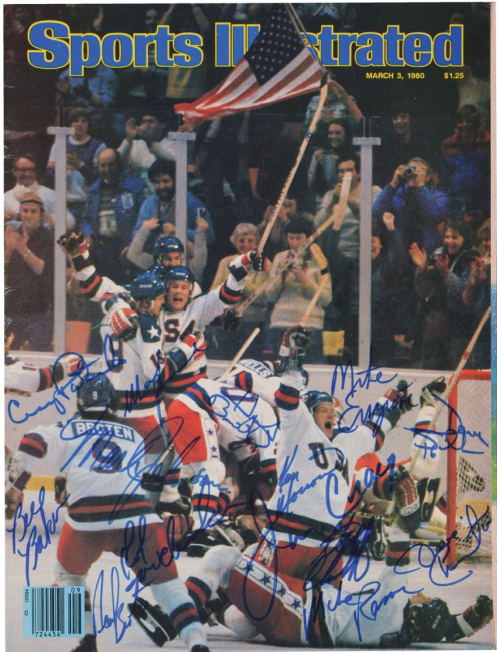
More Rules

- No spectators will be permitted access to the competition. Only registered students and coaches may attend the event.
- Breakfast, lunch, and drinks will be provided. Dietary and allergic restrictions cannot be accommodated. You may bring your own breakfast, lunch, and water bottle as needed.
- Each team (not person) can bring only one laptop
 - A limited number of loaner laptops are available. If your team needs a laptop, please contact Michelle Begonia (codequest-orlando.gr-ebs@lmco.com).
 - **For security reasons, laptops and other devices manufactured by Lenovo are not permitted at this facility. Please do not bring any Lenovo devices with you.**
- All teams must RETURN problem packet to designated box prior to exiting the facility.
- Coaches and students should not post Code Quest competition questions and/or answers online. Questions will be made available on Code Quest Academy after the competition.



Code Quest Internship Program

- Link: <https://lmt.co/4cigpeP>
- High School students, 16 years or older
- US Citizens
- App deadline is March 5
- To strengthen student applications:
 - Include a resume and cover letter.
 - Tailor resumes to the job posting, focusing on applicable qualifications and keywords.
 - Proofread all materials before submitting.
 - Highlight relevant skills, experiences, and projects.
 - Apply only to the posting for the region where the student resides.



Hockey History 2026



Hacker History by Vertasium!



Old Hacker History Slides from 2024...



XZ Utils is a collection of open-source tools and libraries for the XZ compression format, that are used for high compression ratios with support for multiple compression algorithms, notably LZMA2.



On Friday 29th of March, Andres Freund (principal software engineer at Microsoft) emailed oss-security informing the community of the discovery of a backdoor in xz/liblzma version 5.6.0 and 5.6.1.



Github Activity Summary (user: JiaT75)

Repository:
<https://github.com/tukaani-project/xz>

JiaT75's **first commit**
to the XZ repo

PR opened in oss-fuzz to
disable ifunc for fuzzing
builds. Allegedly to mask the
malicious changes.

Obfuscated/encrypted stages binary backdoor
hidden in two test files:

- tests/files/bad-3-corrupt_lzma2.xz
- tests/files/good-large_compressed.lzma.



2021

User **Jia Tan (JiaT75)**
creates his Github Account

2022-02-06

2023-06-28

Potential infrastructure testing:
liblzma: "Add ifunc implementation
to crc64_fast.c."

2023-07-08

2024-02-16

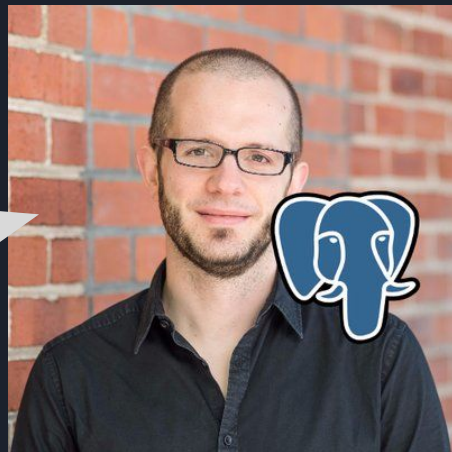
Malicious "build-to-host.m4" file added
to .gitignore, later incorporated to the
package release.

2024-03-09



xz/libzma
v5.6.0 & v5.6.1

Packaged in the final releases



Jia Tan has over 700
commits to xz project

My largest projects have
about 200 commits



m4/build-to-host.m4

The M4 macro is executed during the build process and runs the malicious code below.

```
...
63 gl_[$1]_config='sed \"r\n\" $gl_am_configmake |
eval $gl_path_map | $gl_[$1]_prefix -d 2>/dev/null'
...
95 gl_path_map='tr \"t \t- \" \"t \t-\"'
...
```

Read Bytes

tests/files/bad-3-corrupt_lzma2.xz

Substitution to uncorrupt malformed XZ file

- 0x09 (\t) are replaced with 0x20
- 0x20 (whitespace) are replaced with 0x09
- 0x2d (-) are replaced with 0x5f
- 0x5f (_) are replaced with 0x2d



Stage 1 - Bash File

v5.6.0

- Bytes in comment: 86 F9 5A F7 2E 68 6A BC
- Custom substitution (byte value mapping)

v5.6.1

- Bytes in comment: E5 55 89 B7 24 04 D8 17
- Check if script running on Linux
- Custom substitution (byte value mapping)

tests/files/good-large_compressed.lzma

1. Decompress the file with `xz -dc`
2. Remove junk data from the file using multiple `head` tool calls
3. Portion of the file is discarded (contains the binary backdoor)
4. Use custom substitution cipher to decipher the data
5. Deciphered data is decompressed using `xz -F raw --lzma1 -dc`



Stage 2 - Bash File

v5.6.0 Backdoor extraction

An .o file extracted & integrated into compilation/linking

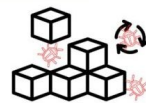
1. Extract & decipher `tests/files/good-large_compressed.lzma`
2. Manipulate output with: `LC_ALL=C sed "s/(\t)/\n/g"`
3. Decrypt using `AWK` script (RC4-like)
4. Decompress with `xz -dc --single-stream`
5. Binary backdoor stored as `liblzma_la-crc64-fast.o`

liblzma_la-crc64-fast.o is then added to the compilation/linking process!



v5.6.1 Extension Mechanism

1. Search Files: use `grep -broaF` in `tests/files/` for signatures:
output: `"file_name:offset:signature"`
 - a. `"~!:_W", "|_|{-"`
 - b. `"jv!^%", "%R,1Z"`
2. If Found:
 - a. Save first offset + 7 as \$start
 - b. Save second file's offset as \$end
3. Next Steps:
 - a. Merge found segments
 - b. Decipher with custom byte mapping
 - c. Decompress & execute data



No files with the signatures were found, however it highlights the framework's potential modularity for future updates

@FRØGGER_
THOMAS ROCCIA

Who would ever try to decipher a "corrupt" file used for testing?

[illegible]

#00Z0.hj0

```
eval `grep ^srcdir= config.status`
```

```
if test -f ../../config.status;then
```

```
eval `grep ^srcdir= ../../config.status`
```

```
srcdir="../../$srcdir"
```

fi

[illegible]

```
####World####
```




Compilation Process

./configure

- **Build-to-Host.m4** is executed and subsequent scripts.
- Target OS check:
 - Ensure it is x86-64 Linux
- Build Context Check:
 - Checks whether is part of a Debian or RPM package build.
- Generate ***malicious*** MakeFile if previous conditions are met.



These conditions targets systems building the xz utility for distribution packages, increasing the chance of spreading the backdoor via official channels.

\$builddir/src/liblzma/MakeFile



Backup original object files

.libs/liblzma_la-crc32_fast.o
.libs/liblzma_la-crc64_fast.o

Specific Compilation Flags:



-Wl, now, -z: Triggers LD_BIND_NOW, making GNU ifunc resolvers run at startup, and gets the backdoor called during this.
-fno-lto: Disables link-time optimization to prevent the compiler and linker from optimizing out the malicious code.



Modification of files **crc64_fast.c** and **crc32_fast.c**.



Compilation, Linking Stage Manipulations, Cleanup

Malicious **liblzma_la-crc64-fast.o** is incorporated into compiled liblzma.



Malicious liblzma_la-crc64-fast.o is incorporated into compiled liblzma.

Exploiting ifunc for Backdoor Execution

GNU indirect functions (ifuncs) is a mechanism allowing the resolution of a function call to different implementations at runtime, occurring very early in the program's startup, before the main application logic begins.

Audit Hook Dynamic Modification

The audit hook alters dynamic symbol resolutions, letting attacker redirects function 'ASA_public_decrypt' to malicious implementation by modifying entries in the Global Offset Table (GOT) which maps code symbols to their absolute memory addresses.



This allows dynamic call behavior modification post-program start and after initial linking.



While ifunc provides the mechanism for conditional function redirection, the audit hook offers the visibility and control over the dynamic linking process needed to trigger the backdoor under precise circumstances.

Backdoored
liblzma



Obfuscated Code Execution

Uses obfuscated code and removes symbol names to complicate static and dynamic analysis.



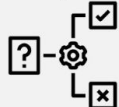
Custom Allocator

Employs a custom allocator to masquerade allocation requests disguised through liblzma's allocation functions, to perform symbol lookup.



Dynamic Analysis Countermeasures

Detects debugging attempts and can alter its behavior to avoid analysis, including disabling itself when certain debugging flags are set.



Conditional Activation

The backdoor selectively initializes only during lzma_crc64 processing by utilizing a call counter within a customized _get_cpuid function, to ensure stealthy activation.

Remote Code Execution via SSH of encrypted message embedded into cryptographic data

Exploitation

2

The `_get_cpuid` function triggers backdoor setup in `lzma_crc64`, and uses a `call count` to ensure activation at the right moment.

3

The modified `RSA_public_decrypt` function authenticates a signature using a `predetermined Ed448 key` before passing a payload to `system()`.

4

The `payload` is decrypted using `ChaCha20`, with the first 32 bytes of the `ED448 public key`.

5

The attacker command is executed allowing **Remote Code Execution**

1

The backdoor is activated by connecting with an **SSH certificate** containing a **payload** in the **Certificate Authority signing key's modulus (N) value**.

libzma is loaded



Hardcoded ED448 public key for signature validation and decrypting the payload.

```
0a 31 fd 3b 2f 1f c6 92 92 68 32 52 c8 cl ac 28
34 d1 f2 c9 75 c4 76 5e b1 f6 88 58 88 93 3e 48
10 0c b0 6c 3a be 14 ee 89 55 d2 45 00 c7 7f 6e
20 d3 2c 60 2b 2c 6d 31 00
```



Attacker

 @FROGGER_
THOMAS ROCCIA



Impacts - Supply Chain Attacks

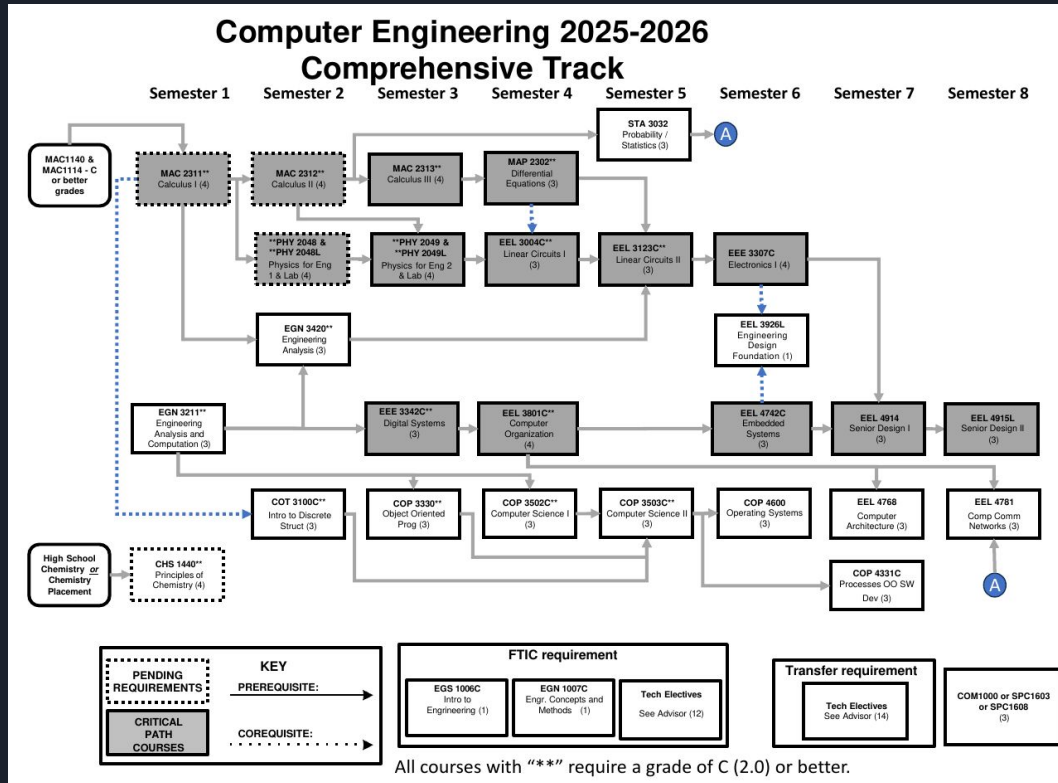
- XZ has a single maintainer - Lasse Collin
 - Unpaid / hobby project for him
 - Development slowed because he was having mental health issues
 - Jai Tan and several other “users” submit commits / pressure Lasse to merge new work
 - By July 2023 Jai Tan has gained Lasse Collin’s trust, is merging commits himself
 - Several other accounts / emails ask for malicious features to get merged
-
- What kind of attacks / methods used?
 - What could have happened had this not been caught?
 - Jai Tan tried to get into Ubuntu for 24.04 LTS release / “important fix”
 - Was trying to get “fixes” in Linux kernel as well



Undergraduate Catalog

- University Education in constant flux
 - New technologies replace older technologies
 - Engineering and Computer Science especially
- Graduation requirements outlined in Undergrad Catalog
 - Published every year / subject to change
 - You get locked into catalog for your enrollment year
 - You can change to newer catalog, but never backwards
- Buy physical or download PDF of your Undergraduate Catalog
 - Know the requirements of your major
 - Talk to your department
 - Be a little wary of general academic advisors

UCF Computer Engineering



Catalog Pages

General Education (GEP)

Complete all of the following

The UCF General Education Program (GEP) is described in this catalog. Engineering students should closely study the requirements of the UCF GEP and the allowable substitutions detailed in paragraphs A through E below to minimize excess hours. Students transferring to UCF from within the Florida College System or State University System should complete the GEP and the Common Program Prerequisites before transferring.

Communication Foundations

Complete all of the following

Required:

Complete the following:

ENC1101 - Composition I (3)

ENC1102 - Composition II (3)

Suggested:

Complete at least 1 of the following:

SPC1603C - Fundamentals of Technical Presentations (3)

SPC1608 - Fundamentals of Oral Communication (3)

Cultural and Historical Foundations

Complete all of the following

Earn at least 6 credits from the following types of courses:

Historical Foundations

Earn at least 3 credits from the following types of courses:

Cultural Foundations

Mathematical Foundations

Complete all of the following

Required:

Complete the following:

MAC2311C - Calculus with Analytic Geometry I (4)

STA3032 - Probability and Statistics for Engineers (3)

Social Foundations

Complete all of the following

Preferred:

Complete the following:

ECO2013 - Principles of Macroeconomics (3)

ECO2023 - Principles of Microeconomics (3)

Or

Select one class from Social Foundations Credit Hours: 3

Science Foundations

Complete all of the following



Required:

Earn at least 4 credits from the following course sets:

keyboard_arrow_up

PHY 2048C

PHY2048C - General Physics Using Calculus I Studio (4)

keyboard_arrow_up

PHY 2048 and 2048L

PHY2048 - General Physics Using Calculus I (3)

PHY2048L - General Physics Using Calculus I Laboratory (1)

Earn at least 3 credits from the following course sets:

keyboard_arrow_up

GEP 11- Science Foundation

AST2002 - Astronomy (3)

CHM1020 - Concepts in Chemistry (3)

CHM1032 - General Chemistry (3)

CHM2045C - Chemistry Fundamentals I (4)

CHS1440C - Principles of Chemistry (4)

PHY1038 - Physics of Energy, Climate Change and Environment (3)

PHY2020 - Concepts of Physics (3)

PHY2048 - General Physics Using Calculus I (3)

PHY2053 - College Physics I (3)

Common Program Prerequisites (CPP) (If applicable)

Complete all of the following

These courses are specifically required for all engineering students of the Florida State University System. CPP courses are also available at other Florida post-secondary schools and may be transferred directly to UCF programs. To enroll in CpE major courses, a 2.0 (C or better) in each course is required.

See "Common Prerequisites" in the Transfer and Transitions Services section for more information.

Earn at least 15 credits from the following:

MAC2311C - Calculus with Analytic Geometry I (4)

MAC2312 - Calculus with Analytic Geometry II (4)

MAC2313 - Calculus with Analytic Geometry III (4)

MAP2302 - Ordinary Differential Equations I (3)

PHY 2048C - General Physics Using Calculus I (or PHY2048 and PHY2048L) Credit Hours: 4 (GEP)

PHY 2049C - General Physics Using Calculus II (or PHY2049 and PHY2049L) Credit Hours: 4

GEP Courses: MAP 2311C, PHY 2048C/PHY2048 & PHY 2048L

Complete at least 1 of the following:

CHS1440C - Principles of Chemistry (4)

CHM2045C - Chemistry Fundamentals I (4)

* Preferred: CHS 1440

Grand Total Credits: **19**



Computer Engineering

- General Education (57 hrs)
 - 38 hr general education
 - 19 hrs common pre-reqs
- Degree Requirements (71 hrs)
 - 51 hrs required CS/CpE
 - 12 hrs tech electives
 - 6 hrs senior design
- 128 credit hours

Courses Required for the Major

51 Total Credits

keyboard_arrow_up

Complete all of the following

Complete the following:

COT3100C - Introduction to Discrete Structures (3)
COP3502C - Computer Science I (3)
COP3503C - Computer Science II (3)
COP3330 - Object Oriented Programming (3)
COP4331C - Processes for Object-Oriented Software Development (3)
EEL3926L - ECE Design Lab (1)
EEL3004C - Linear Circuits I (3)
EEL3123C - Linear Circuits II (3)
EEE3307C - Electronics I (4)
EEE3342C - Digital Systems (3)
EEL3801C - Computer Organization (4)
EEL4742C - Embedded Systems (3)
EEL4768 - Computer Architecture (3)
EEL4781 - Computer Communication Networks (3)
COP4600 - Operating Systems (3)

EGN3211 - Engineering Analysis and Computation (3)

EGN3420 - Engineering Analysis (3)

A "C" (2.0) or better is required in the following courses: COP 3330, COP 3502C, COP 3503C, COT 3100C, EEE 3342C, EEL 3004C, EEL 3123C, EEL 3801C, EGN 3211, EGN 3420

Restricted Electives

12 Total Credits

keyboard_arrow_up

Technical electives are available in the BSCpE program to address specific student interests in a variety of technical areas such as Software Engineering. Students should consult with their academic advisor for the identification of courses that are approved technical electives and the terms when specific courses of

Capstone Requirements

6 Total Credits

keyboard_arrow_up

Complete the following:

EEL4914 - Senior Design I (3)
EEL4915L - Senior Design II (3)

Grand Total Credits: **71**

Computer Science

- General Education (56 hrs)
 - 42 hr general education
 - 14 hrs common pre-reqs
- Degree Requirements (60 hrs)
 - 30 hrs required CS
 - 18 hrs tech electives
 - 6 hrs advanced math
 - 6 hrs senior design
- 120 credit hours

Cybersecurity Area

CAP 4145 - Introduction to Malware Analysis **Credit Hours: 3**
CIS 3362 - Cryptography and Information Security **Credit Hours: 3**
CIS 4203C - Digital Forensics **Credit Hours: 3**
CIS 4361 - Secure Operating Systems and Administration **Credit Hours: 3**
CIS 4615 - Secure Software Development and Assurance **Credit Hours: 3**
CIS 4940C - Topics in Cybersecurity **Credit Hours: 3**
CNT 4403 - Network Security and Privacy **Credit Hours: 3**
EEE 4346C - Hardware Security and Trusted Circuit Design **Credit Hours: 3**

Degree Requirements

Core Requirements: Basic Level
30 Total Credits
keyboard_arrow_up

Complete all of the following

Earn at least 27 credits from the following:

STA2023 - Statistical Methods I (3)
COP3330 - Object Oriented Programming (3)
COP3502C - Computer Science I (3)
COP3503C - Computer Science II (3)
CDA3103C - Computer Logic and Organization (3) 
COT3100C - Introduction to Discrete Structures (3)
CIS3360 - Security in Computing (3) 
COP3402 - Systems Software (3) 
COT4210 - Discrete Structures II (3) 
COP4331C - Processes for Object-Oriented Software Development (3) 
COT3960 - Foundation Exam 

Complete at least 1 of the following: 

ENC3241 - Writing for the Technical Professional (3)
ENC3250 - Professional Writing (3)

Grand Total Credits: **30**

Program Details

Core Requirements: Advanced Level (18 Credit Hours)

AI and Machine Learning Area

CAP 4053 - AI for Game Programming **Credit Hours: 3**
CAP 4453 - Robot Vision **Credit Hours: 3**
CAP 4611 - Algorithms for Machine Learning **Credit Hours: 3**
CAP 4630 - Artificial Intelligence **Credit Hours: 3**
CAP 5415 - Computer Vision **Credit Hours: 3**
CAP 5512 - Evolutionary Computation **Credit Hours: 3**
CAP 5610 - Machine Learning **Credit Hours: 3**
CAP 5636 - Advanced Artificial Intelligence **Credit Hours: 3**



UF Computer Engineering

Degree Reqs (56 hrs total)

- 38 hrs CpE Reqs
- 12 hrs Tech Electives
- 6 hrs Senior Design

126 total credit hrs required

Computer Engineering Requirements

A minimum grade of C is required for each critical-tracking course and the critical-tracking GPA must be a minimum of 2.5.

A minimum grade of C is required in any computer engineering course that is a prerequisite for another computer engineering course and CpE Design 2 CEN 4908C. The prerequisite course and its subsequent course cannot be taken the same term, even if the prerequisite course is being repeated.

Minimum grades of C are required in:

Code	Title	Credits
CDA 3101	Introduction to Computer Organization	3
CEN 3031	Introduction to Software Engineering	3
COP 3502C	Programming Fundamentals 1	4
COP 3503C	Programming Fundamentals 2	4
COP 3504C	Advanced Programming Fundamentals for CIS Majors	4
COP 3530	Data Structures and Algorithm	3
COT 3100	Applications of Discrete Structures	3
EEL 3701C	Digital Logic and Computer Systems	4
EEL 4744C	Microprocessor Applications	4
ENC 3246	Professional Communication for Engineers	3
CpE Design 1		
Select one:		3
CEN 3907C	Computer Engineering Design 1	
EGN 4951	Integrated Product and Process Design 1	
CpE Design 2		
Select one:		3
CEN 4908C	Computer Engineering Design 2	
EGN 4952	Integrated Product and Process Design 2	

Electrical Engineering

- 6 classes changed from computer engineering
- These are all intensely challenging

Courses Required for the Major

47 Total Credits

keyboard_arrow_up

Complete all of the following

Complete the following:

EEL3926L - ECE Design Lab (1)
EGN3211 - Engineering Analysis and Computation (3)
EEL3004C - Linear Circuits I (3)
EEL3123C - Linear Circuits II (3)
EEE3350 - Semiconductor Devices (3) ←
EEE3307C - Electronics I (4)
EEE3342C - Digital Systems (3)
EEL3470 - Electromagnetic Fields (3) ←
EEL3552C - Signal Analysis and Analog Communication (4) ←
EEL3657 - Linear Control Systems (3) ←
EEL3801C - Computer Organization (4)
EEE4309C - Electronics II (4) ←
EEL4750 - Digital Signal Processing Fundamentals (3) ←
EEL4742C - Embedded Systems (3)
EGN3420 - Engineering Analysis (3)

A "C" (2.0) or better is required in the following courses: EEE 3342C, EEL 3004C, EEL 3123C, EEL 3801C, EGN 3211, EGN 3420

Restricted Electives

16 Total Credits

keyboard_arrow_up

Technical electives are available in the BSEE program to address specific student interests in a variety of technical areas. Students should consult with their academic advisor for the identification of courses that are approved technical electives and the terms when specific courses of this type are to be offered.



Links

•
•